

Spring Cloud & Microservices Cheat Sheet

Core Spring Cloud Components

Service Discovery (Eureka)

Server Setup:

```
java

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Client Registration:

```
java

@SpringBootApplication
@EnableEurekaClient // or @EnableDiscoveryClient
public class ServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ServiceApplication.class, args);
    }
}
```

Configuration (application.yml):

yaml

Eureka Server

```
eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
```

Eureka Client

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
```

API Gateway (Spring Cloud Gateway)

Basic Gateway Configuration:

java

```
@SpringBootApplication
public class GatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("user-service", r -> r.path("/api/users/**")
                .uri("lb://user-service"))
            .route("order-service", r -> r.path("/api/orders/**")
                .uri("lb://order-service"))
            .build();
    }
}
```

YAML Configuration:

yaml

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=2
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
```

Load Balancing (Ribbon/LoadBalancer)

Using RestTemplate:

java

@Configuration

```
public class RestTemplateConfig {
```

```
    @Bean
```

```
    @LoadBalanced
```

```
    public RestTemplate restTemplate() {
```

```
        return new RestTemplate();
```

```
    }
```

```
}
```

// Usage

@Service

```
public class UserService {
```

```
    @Autowired
```

```
    private RestTemplate restTemplate;
```

```
    public User getUser(Long id) {
```

```
        return restTemplate.getForObject(  
            "http://user-service/users/" + id, User.class);
```

```
    }
```

```
}
```

Using WebClient:

```
java
```

```
@Configuration
```

```
public class WebClientConfig {  
  
    @Bean  
    @LoadBalanced  
    public WebClient.Builder webClientBuilder() {  
        return WebClient.builder();  
    }  
}
```

```
// Usage
```

```
@Service
```

```
public class UserService {  
  
    private final WebClient webClient;  
  
    public UserService(WebClient.Builder webClientBuilder) {  
        this.webClient = webClientBuilder.build();  
    }  
  
    public Mono<User> getUser(Long id) {  
        return webClient.get()  
            .uri("http://user-service/users/{id}", id)  
            .retrieve()  
            .bodyToMono(User.class);  
    }  
}
```

Circuit Breaker (Hystrix/Resilience4j)

Hystrix (Legacy):

java

@Service

```
public class UserService {

    @HystrixCommand(fallbackMethod = "getDefaultUser")
    public User getUser(Long id) {
        return restTemplate.getForObject("http://user-service/users/" + id, User.class);
    }

    public User getDefaultUser(Long id) {
        return new User(id, "Default User");
    }
}

@SpringBootApplication
@EnableCircuitBreaker
public class Application {
    // ...
}
```

Resilience4j (Modern):

java

@Service

```
public class UserService {

    @CircuitBreaker(name = "user-service", fallbackMethod = "getDefaultUser")
    @Retry(name = "user-service")
    @TimeLimiter(name = "user-service")
    public CompletableFuture<User> getUser(Long id) {
        return CompletableFuture.supplyAsync(() ->
            restTemplate.getForObject("http://user-service/users/" + id, User.class));
    }

    public CompletableFuture<User> getDefaultUser(Long id, Exception ex) {
        return CompletableFuture.completedFuture(new User(id, "Default User"));
    }
}
```

Resilience4j Configuration:

```
yaml
```

```
resilience4j:
  circuitbreaker:
    instances:
      user-service:
        registerHealthIndicator: true
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        permittedNumberOfCallsInHalfOpenState: 3
        waitDurationInOpenState: 5s
        failureRateThreshold: 50
  retry:
    instances:
      user-service:
        maxAttempts: 3
        waitDuration: 1s
```

Configuration Management (Spring Cloud Config)

Config Server:

```
java
```

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

Config Server Properties:

yaml

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-org/config-repo
          search-paths: '{application}'
```

Config Client:

yaml

```
spring:
  application:
    name: user-service
  cloud:
    config:
      uri: http://localhost:8888
      fail-fast: true
  config:
    import: configserver:http://localhost:8888
```

Refresh Configuration:

java

```
@RestController
@RefreshScope
public class ConfigController {

    @Value("${message:Default}")
    private String message;

    @GetMapping("/message")
    public String getMessage() {
        return message;
    }
}
```


Microservices Patterns

Service Communication

Synchronous Communication:

```
java

// Feign Client
@FeignClient(name = "user-service")
public interface UserClient {

    @GetMapping("/users/{id}")
    User getUser(@PathVariable Long id);

    @PostMapping("/users")
    User createUser(@RequestBody User user);
}

// Usage
@Service
public class OrderService {

    @Autowired
    private UserClient userClient;

    public Order createOrder(OrderRequest request) {
        User user = userClient.getUser(request.getUserId());
        // Process order
        return new Order();
    }
}
```

Asynchronous Communication (Message Queues):

java

// RabbitMQ Producer

@Component

public class OrderEventPublisher {

 @Autowired

 private RabbitTemplate rabbitTemplate;

 public void publishOrderCreated(Order order) {

 rabbitTemplate.convertAndSend("order.exchange", "order.created", order);

 }

}

// RabbitMQ Consumer

@Component

public class OrderEventListener {

 @RabbitListener(queues = "inventory.queue")

 public void handleOrderCreated(Order order) {

// Update inventory

 }

}

Data Management Patterns

Database per Service:

java

// User Service - MySQL

@Entity

@Table(name = "users")

public class User {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private String name;

 private String email;

}

// Order Service - PostgreSQL

@Entity

@Table(name = "orders")

public class Order {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private Long userId;

 private BigDecimal amount;

}

CQRS Pattern:

java

// Command Side

@Service

```
public class OrderCommandService {  
  
    public void createOrder(CreateOrderCommand command) {  
        Order order = new Order(command);  
        orderRepository.save(order);  
        eventPublisher.publish(new OrderCreatedEvent(order));  
    }  
}
```

// Query Side

@Service

```
public class OrderQueryService {  
  
    public OrderView getOrder(Long id) {  
        return orderViewRepository.findById(id);  
    }  
  
    public List<OrderView> getOrdersByUser(Long userId) {  
        return orderViewRepository.findById(userId);  
    }  
}
```

Saga Pattern:

java

// Orchestration-based Saga

@Service

```
public class OrderSagaOrchestrator {

    public void processOrder(Order order) {
        try {
            // Step 1: Reserve inventory
            inventoryService.reserveItems(order.getItems());

            // Step 2: Process payment
            paymentService.processPayment(order.getPayment());

            // Step 3: Ship order
            shippingService.shipOrder(order);

        } catch (Exception e) {
            // Compensate
            compensateOrder(order);
        }
    }

    private void compensateOrder(Order order) {
        inventoryService.releaseItems(order.getItems());
        paymentService.refundPayment(order.getPayment());
    }
}
```

Observability

Distributed Tracing (Sleuth + Zipkin):

yaml

```
spring:
  sleuth:
    zipkin:
      base-url: http://localhost:9411
    sampler:
      probability: 1.0
```

Custom Spans:

java

@Service

```
public class UserService {

    @Autowired
    private Tracer tracer;

    public User getUser(Long id) {
        Span span = tracer.nextSpan().name("get-user").start();
        try (Tracer.SpanInScope ws = tracer.withSpanInScope(span)) {
            span.tag("user.id", String.valueOf(id));
            return userRepository.findById(id);
        } finally {
            span.end();
        }
    }
}
```

Health Checks:

java

@Component

```
public class DatabaseHealthIndicator implements HealthIndicator {

    @Autowired
    private DataSource dataSource;

    @Override
    public Health health() {
        try {
            dataSource.getConnection().isValid(1);
            return Health.up().withDetail("database", "Available").build();
        } catch (Exception e) {
            return Health.down(e).withDetail("database", "Unavailable").build();
        }
    }
}
```

Metrics (Micrometer):

java

@Service

```
public class OrderService {

    private final MeterRegistry meterRegistry;
    private final Counter orderCounter;

    public OrderService(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
        this.orderCounter = Counter.builder("orders.created")
            .description("Number of orders created")
            .register(meterRegistry);
    }

    public Order createOrder(OrderRequest request) {
        Timer.Sample sample = Timer.start(meterRegistry);
        try {
            Order order = processOrder(request);
            orderCounter.increment();
            return order;
        } finally {
            sample.stop(Timer.builder("order.creation.time")
                .description("Order creation time")
                .register(meterRegistry));
        }
    }
}
```

Security in Microservices

OAuth2 & JWT

Resource Server Configuration:

java

@Configuration

@EnableResourceServer

```
public class ResourceServerConfig extends ResourceServerConfigurerAdapter {
```

```
    @Override
```

```
    public void configure(HttpSecurity http) throws Exception {
```

```
        http.authorizeRequests()
```

```
            .antMatchers("/api/public/**").permitAll()
```

```
            .anyRequest().authenticated();
```

```
    }
```

```
    @Bean
```

```
    public JwtAccessTokenConverter accessTokenConverter() {
```

```
        JwtAccessTokenConverter converter = new JwtAccessTokenConverter();
```

```
        converter.setSigningKey("secret-key");
```

```
        return converter;
```

```
    }
```

```
}
```

Method-level Security:

java

@RestController

@PreAuthorize("hasRole('USER')")

```
public class UserController {
```

```
    @GetMapping("/users/{id}")
```

```
    @PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
```

```
    public User getUser(@PathVariable Long id) {
```

```
        return userService.getUser(id);
```

```
    }
```

```
}
```

Testing Strategies

Contract Testing (Spring Cloud Contract)

Contract Definition (Groovy DSL):

groovy

```
Contract.make {
    description "should return user by id"
    request {
        method GET()
        url "/users/1"
        headers {
            contentType(applicationJson())
        }
    }
    response {
        status OK()
        body([
            id: 1,
            name: "John Doe",
            email: "john@example.com"
        ])
        headers {
            contentType(applicationJson())
        }
    }
}
```

Integration Testing:

java

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@TestPropertySource(properties = {
    "eureka.client.enabled=false",
    "spring.cloud.config.enabled=false"
})
class UserServiceIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void shouldGetUser() {
        ResponseEntity<User> response = restTemplate.getForEntity("/users/1", User.class);
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(response.getBody().getName()).isEqualTo("John Doe");
    }
}
```

Common Configurations

Application Properties Template

yaml

```
server:
  port: 8080

spring:
  application:
    name: user-service
  datasource:
    url: jdbc:mysql://localhost:3306/userdb
    username: user
    password: password
  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: false
  cloud:
    config:
      uri: http://localhost:8888
      fail-fast: true

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always

logging:
  level:
    org.springframework.cloud: DEBUG
    com.yourpackage: INFO
```

Docker Configuration

Dockerfile:

dockerfile

```
FROM openjdk:11-jre-slim
```

```
COPY target/user-service-1.0.0.jar app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Docker Compose:

yaml

```
version: '3.8'
```

```
services:
```

```
  eureka-server:
```

```
    image: eureka-server:latest
```

```
    ports:
```

```
      - "8761:8761"
```

```
  config-server:
```

```
    image: config-server:latest
```

```
    ports:
```

```
      - "8888:8888"
```

```
    depends_on:
```

```
      - eureka-server
```

```
  user-service:
```

```
    image: user-service:latest
```

```
    ports:
```

```
      - "8081:8080"
```

```
    depends_on:
```

```
      - eureka-server
```

```
      - config-server
```

```
    environment:
```

```
      - EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE=http://eureka-server:8761/eureka
```

```
      - SPRING_CLOUD_CONFIG_URI=http://config-server:8888
```

Key Dependencies (Maven)


```
<dependencies>
  <!-- Spring Boot Starter -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Eureka Discovery -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>

  <!-- Gateway -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
  </dependency>

  <!-- Config Client -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>

  <!-- Feign Client -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
  </dependency>

  <!-- Circuit Breaker -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
  </dependency>

  <!-- Tracing -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-sleuth</artifactId>
  </dependency>
```

```
<!-- Zipkin -->  
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-sleuth-zipkin</artifactId>  
</dependency>  
</dependencies>
```

Best Practices

1. **Service Boundaries:** Design services around business capabilities
2. **Data Isolation:** Each service should own its data
3. **Stateless Services:** Keep services stateless for better scalability
4. **Idempotency:** Make operations idempotent where possible
5. **Graceful Degradation:** Implement fallback mechanisms
6. **Monitoring:** Implement comprehensive logging and monitoring
7. **Security:** Secure service-to-service communication
8. **Configuration:** Externalize configuration
9. **Versioning:** Plan for API versioning from the start
10. **Testing:** Implement contract testing between services